

Rapidly-Exploring Dense Tree (RDT)

A dense sequence of samples is used as a guide in the incremental construction of the tree.

If this sequence is random, the resulting tree is called a **rapidly exploring random tree** (RRT).

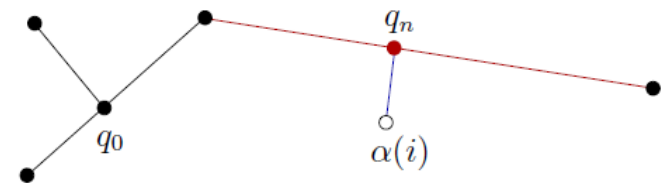
In general, this family of trees, whether the sequence is random or deterministic, will be referred to as rapidly exploring dense trees (RDTs)

Rapidly-Exploring Dense Tree (RDT)

Right now, we **do not** consider obstacles

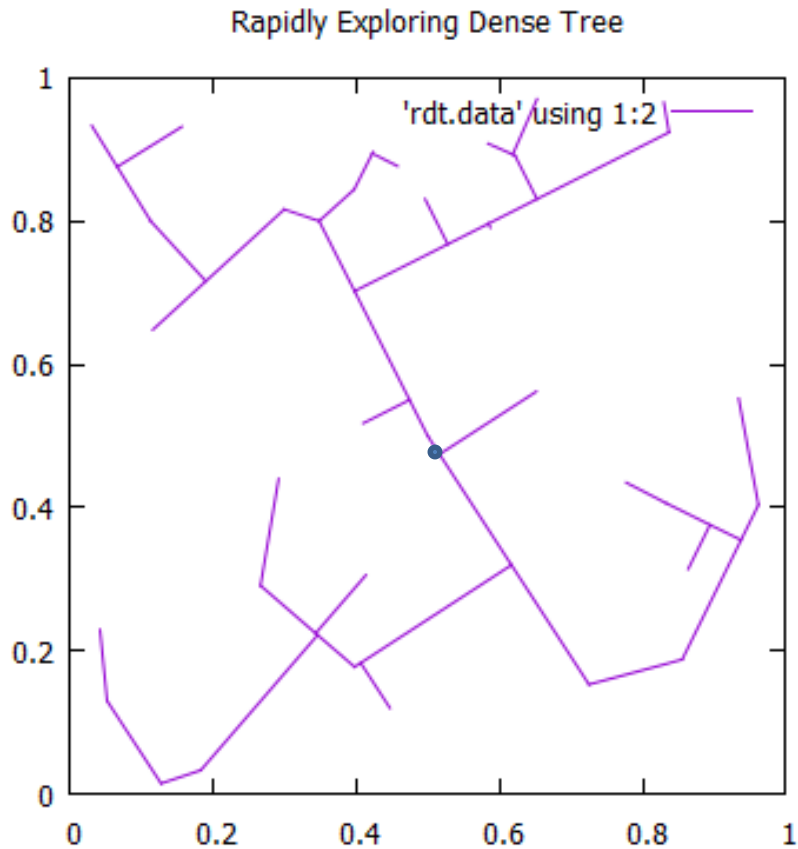
SIMPLE_RDT(q_0)

```
1   $\mathcal{G}.\text{init}(q_0);$   
2  for  $i = 1$  to  $k$  do  
3     $\mathcal{G}.\text{add\_vertex}(\alpha(i));$   
4     $q_n \leftarrow \text{NEAREST}(S(\mathcal{G}), \alpha(i));$   
5     $\mathcal{G}.\text{add\_edge}(q_n, \alpha(i));$ 
```



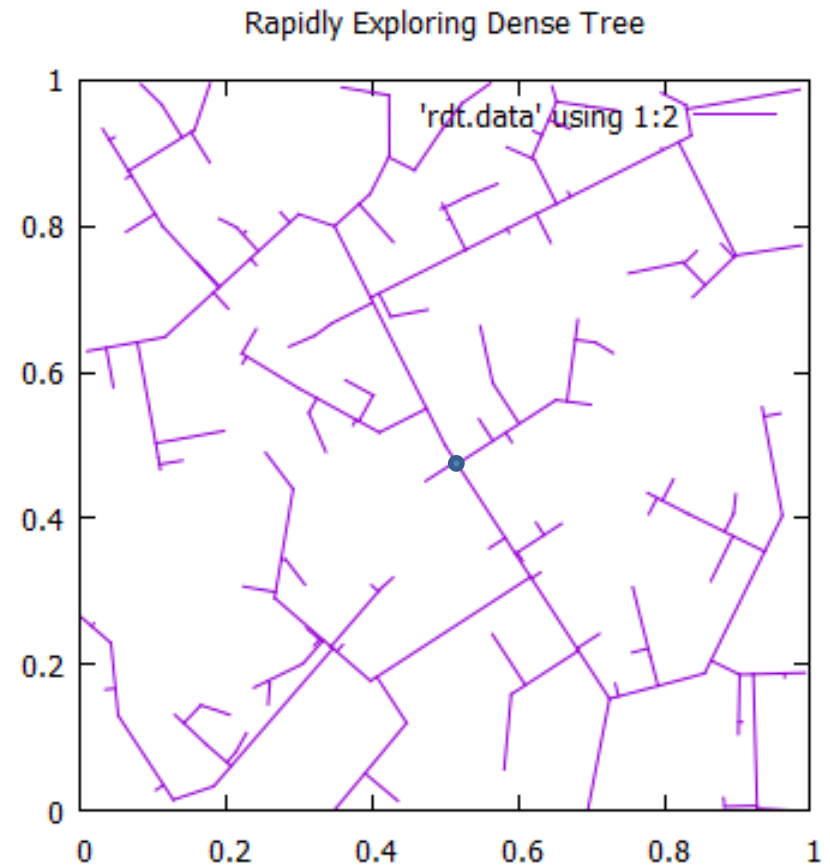
If the nearest point in S lies in an edge, then the edge is split into two, and a new vertex is inserted into G .

Rapidly-Exploring Dense Tree (RDT)



N = 50

Start Point (0.5,0.5)

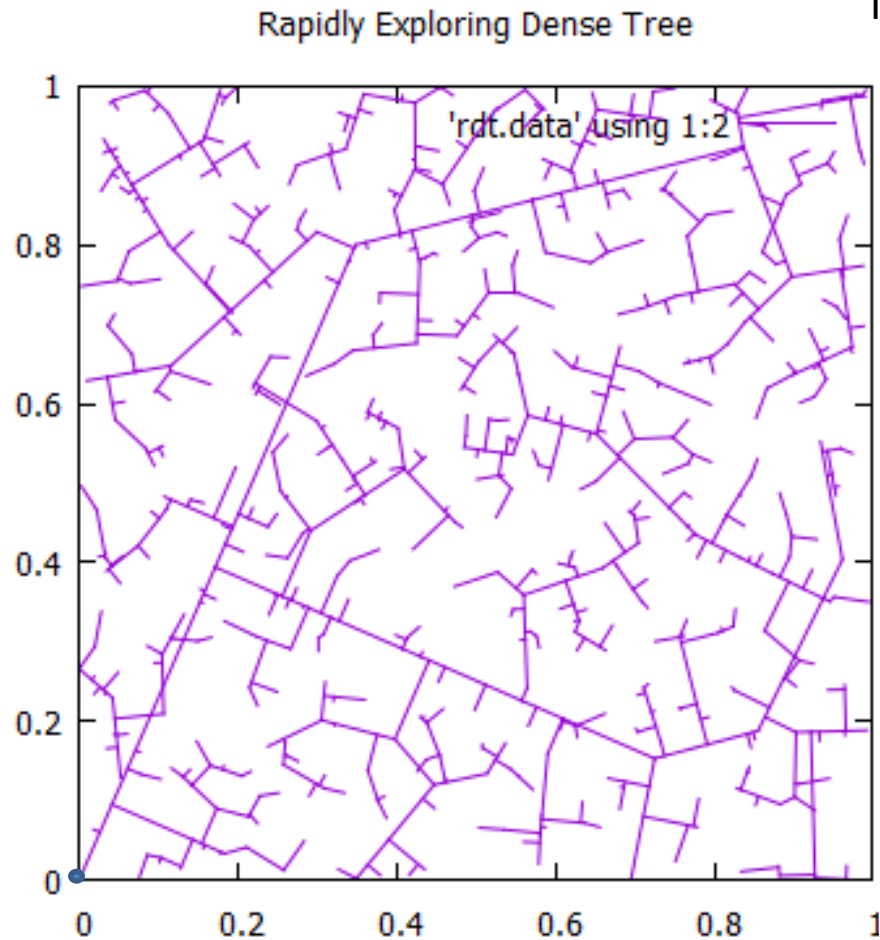


N = 250

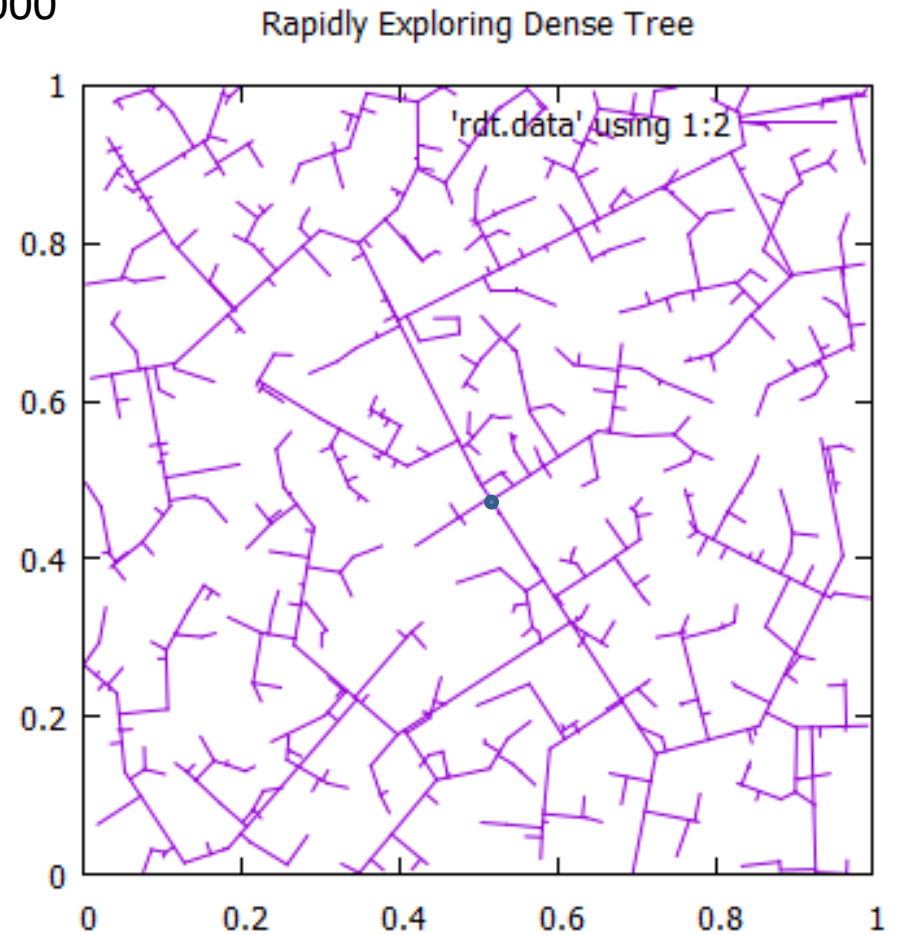
Start Point (0.5,0.5)

Rapidly-Exploring Dense Tree (RDT)

N = 1000

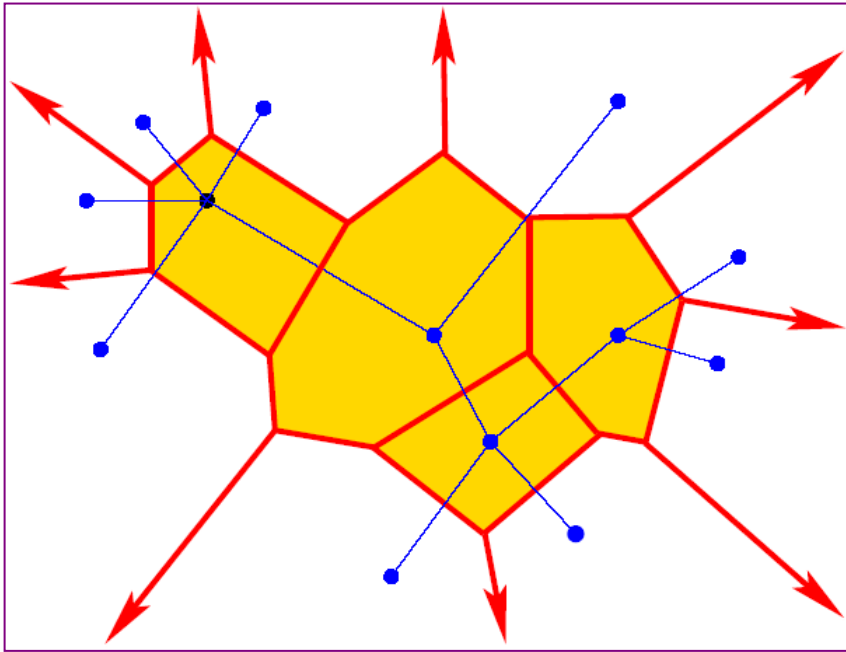


Start Point (0,0)

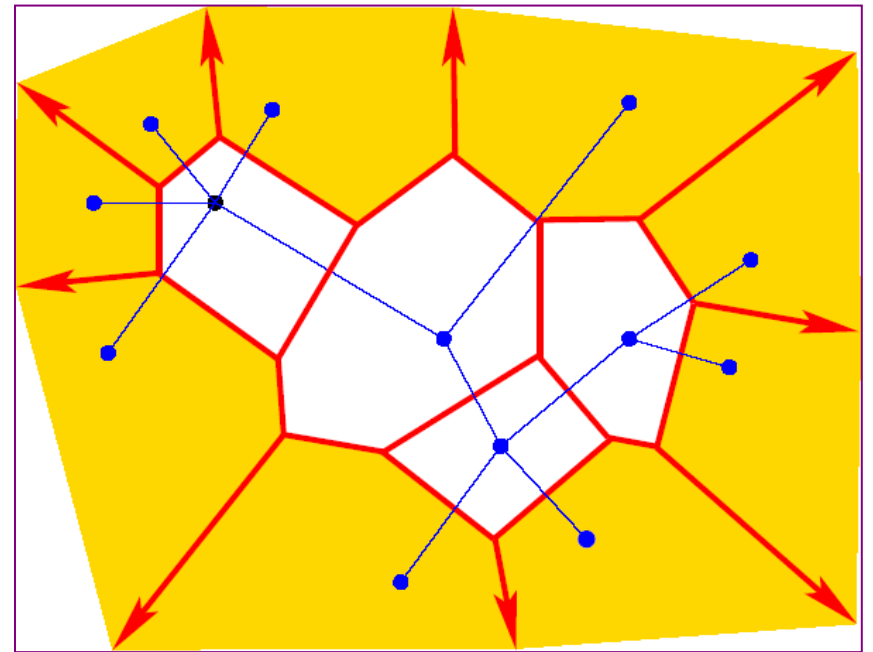


Start Point (0.5,0.5)

Refinement vs. Expansion



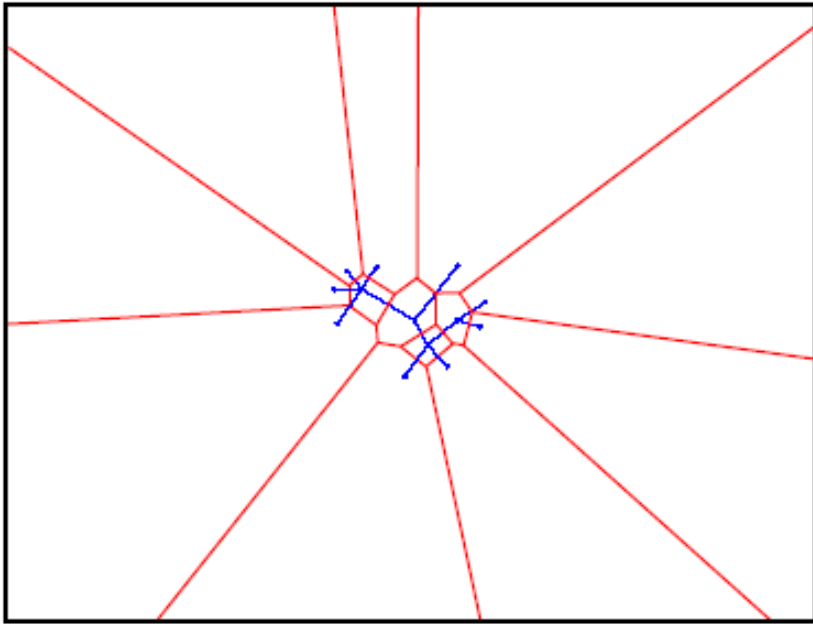
refinement



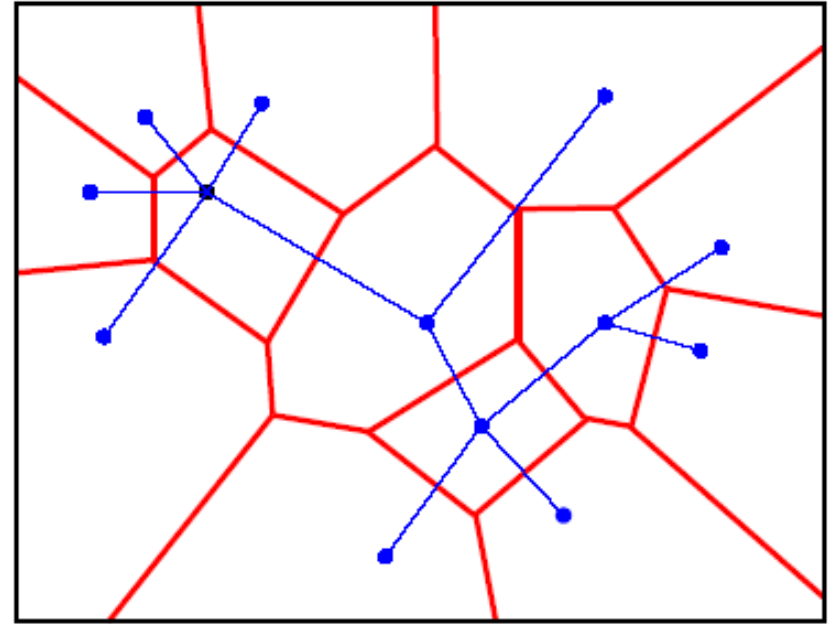
expansion

Where will the random sample fall? How to control the behavior of RRT?

Determining the Boundary



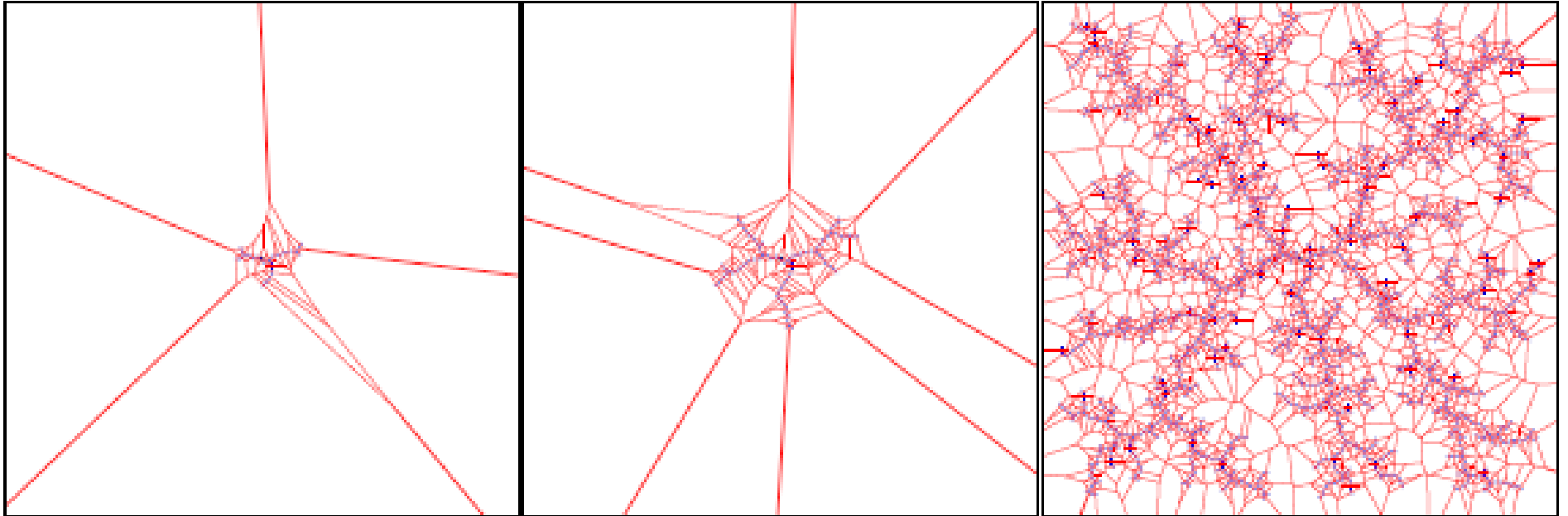
Expansion dominates



Balanced refinement and expansion

The tradeoff depends on the size of the bounding box

Why “Rapidly Exploring”?

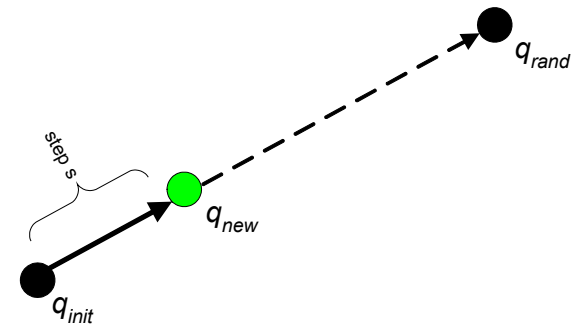


What is the probability that a vertex will be extended?

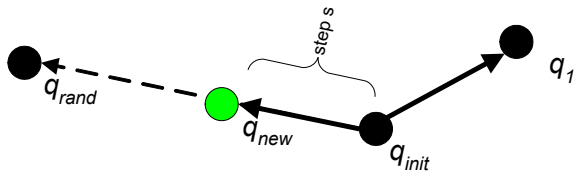
Proportional to the area of its Voronoi region

RRT Algorithm

C-Space

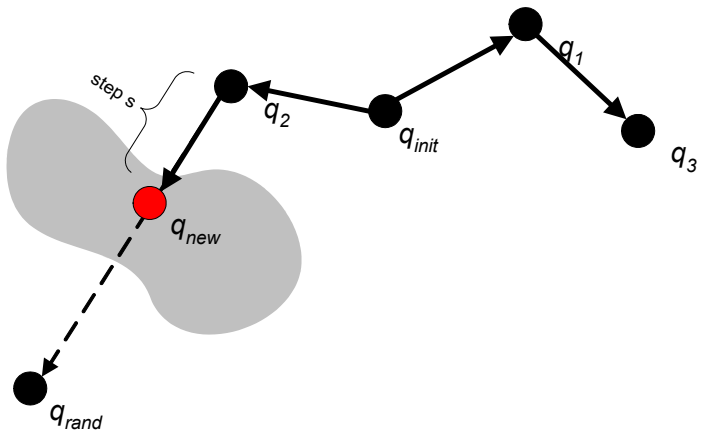


C-Space

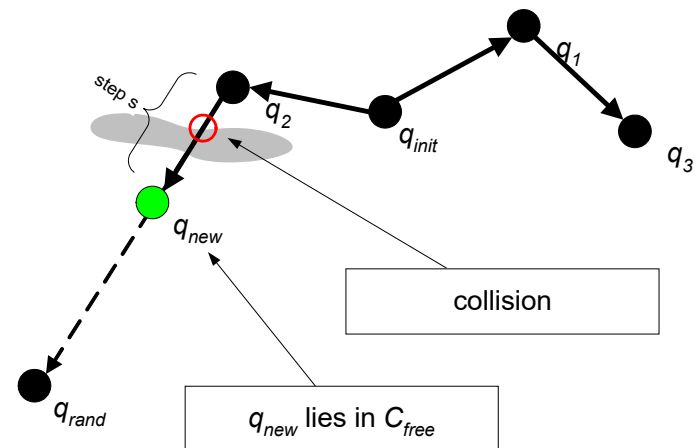


RRT Algorithm

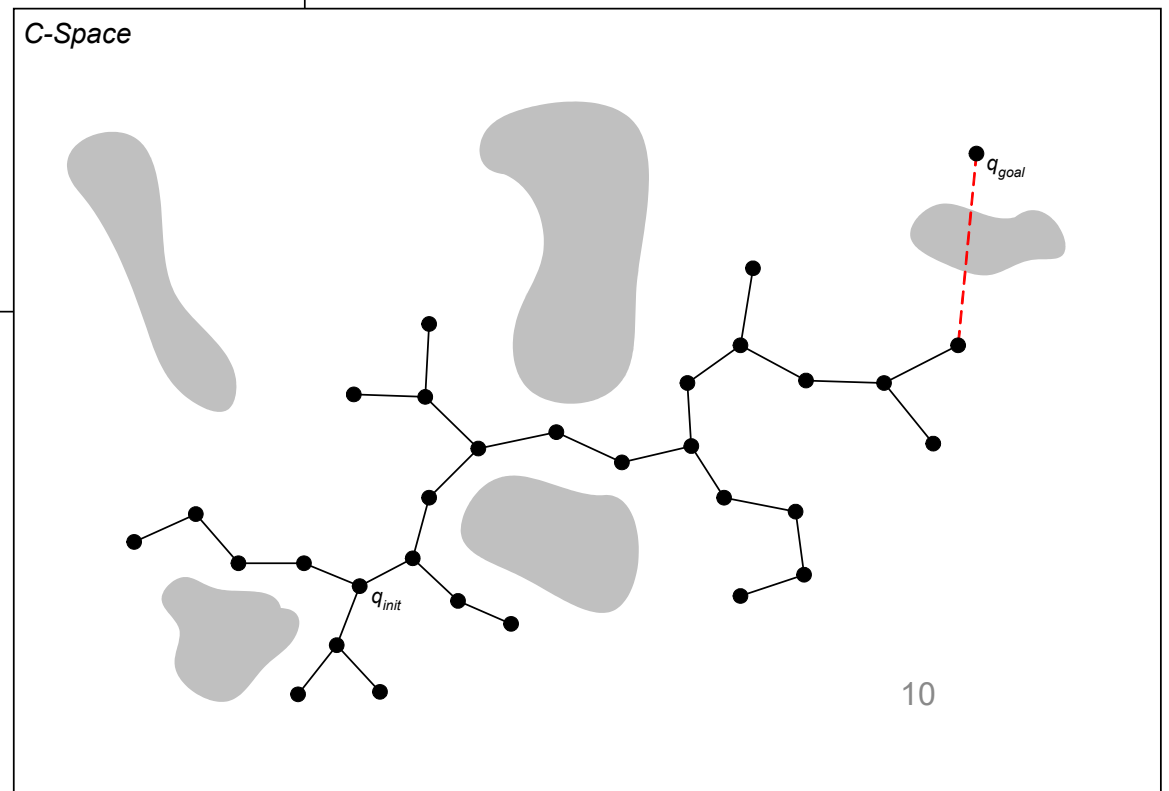
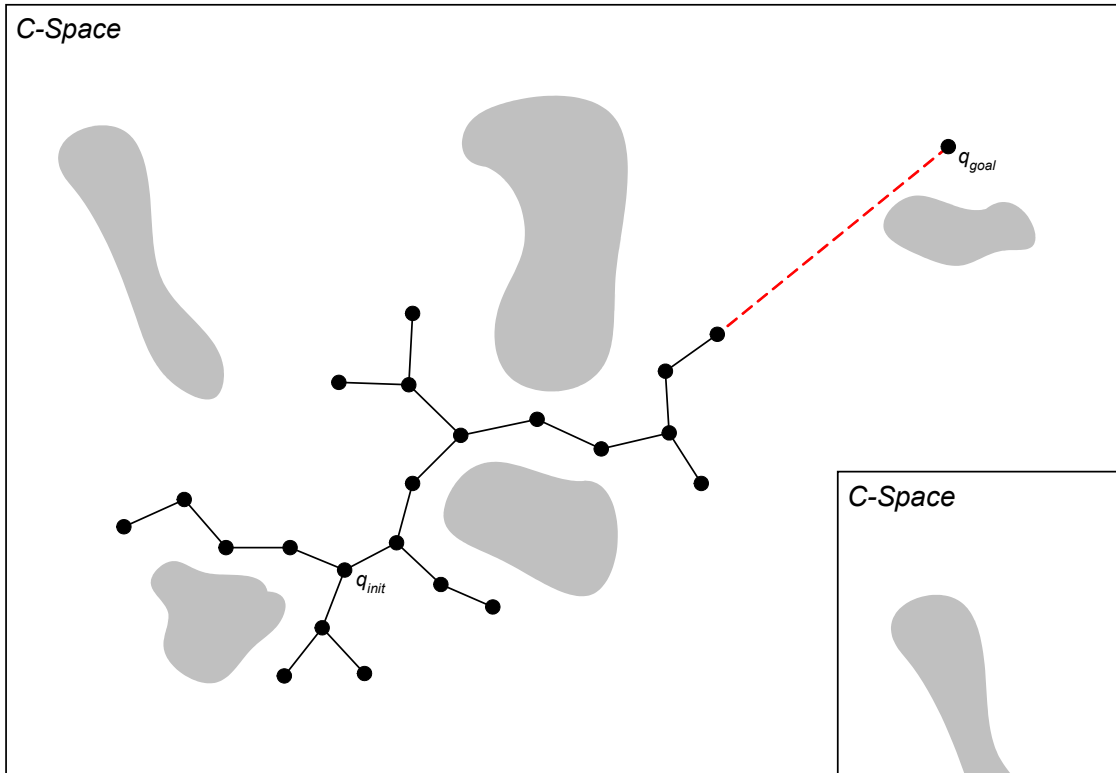
C-Space



C-Space

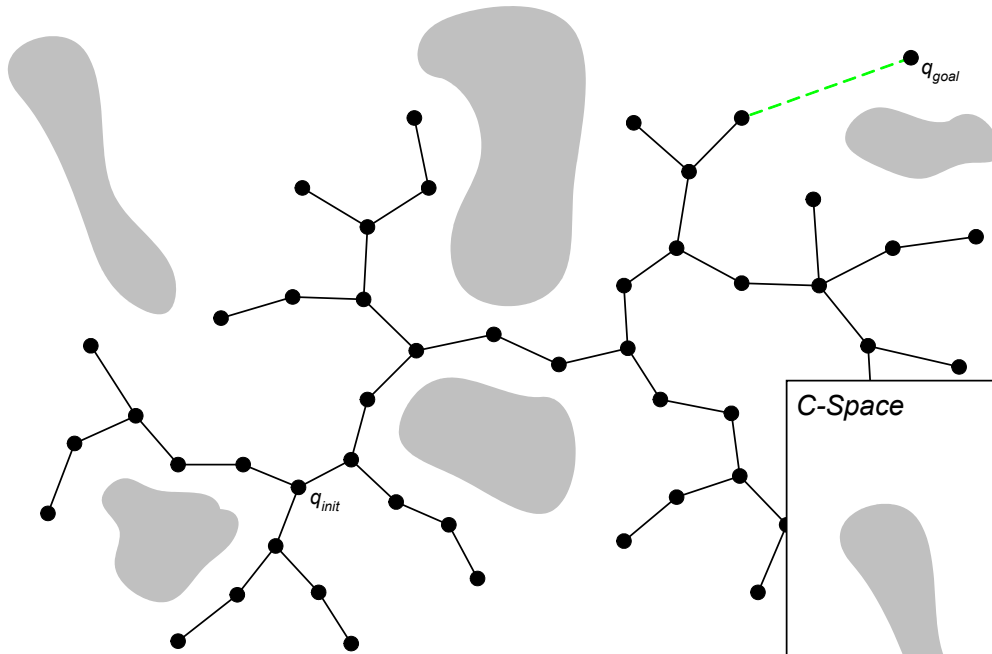


RRT Algorithm

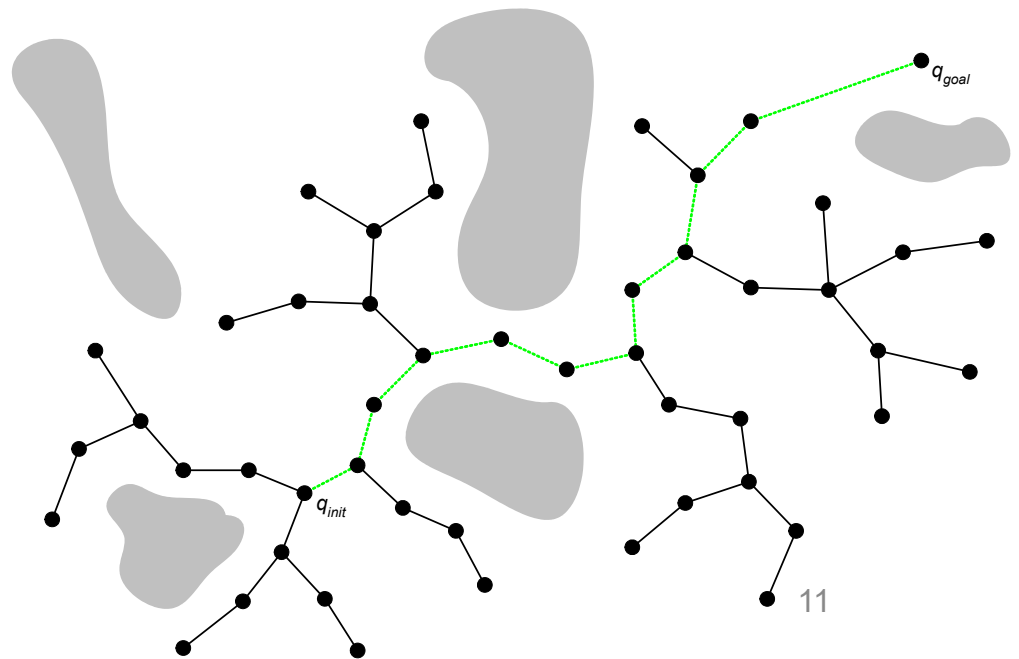


RRT Algorithm

C-Space

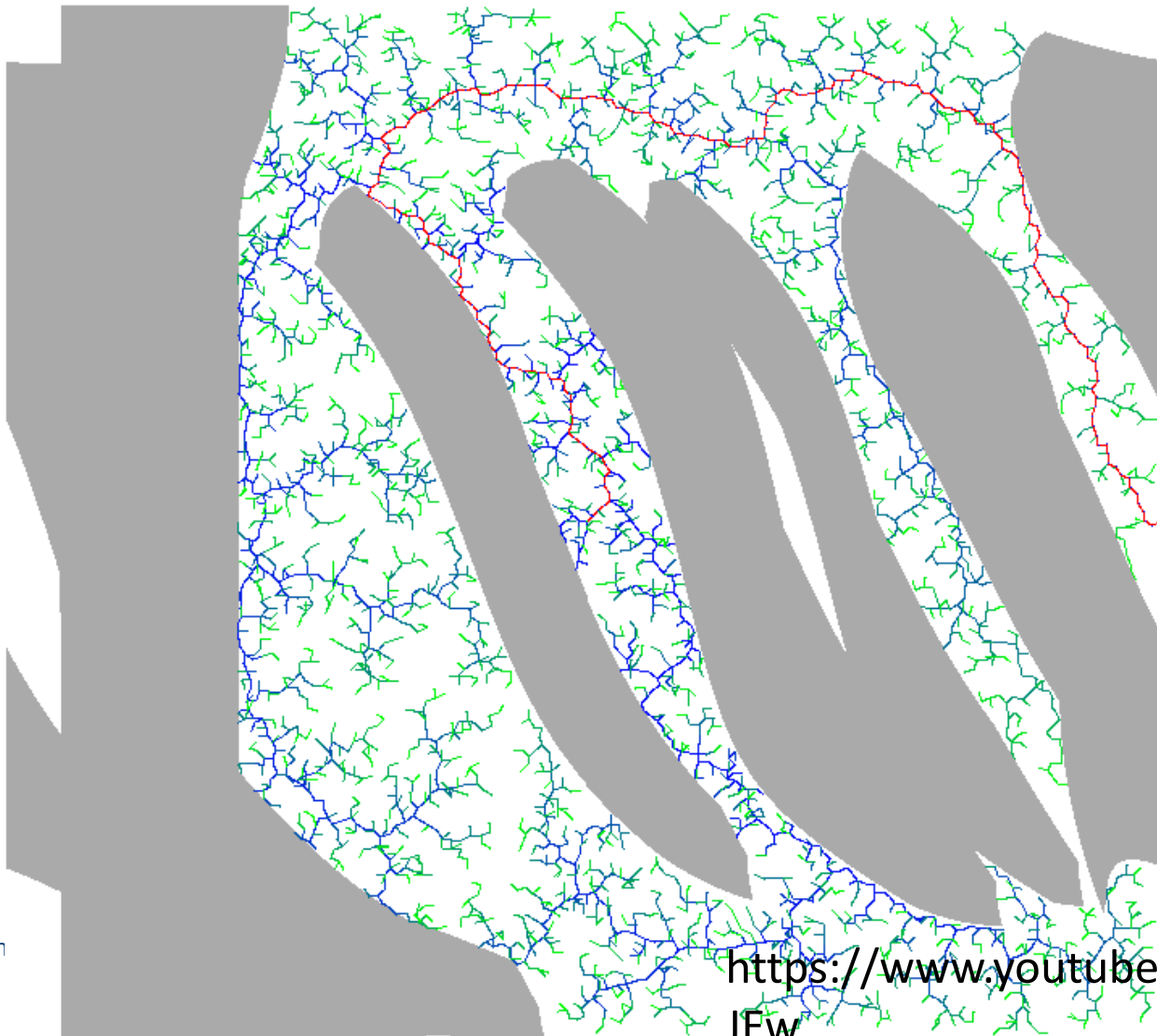


C-Space



RRT Algorithm

colors represent the growth of the RRT over time



Motion Planning with Rapidly-Exploring Random Trees (RRT)

Algorithm 3: Build RRT Algorithm (RRT)

Data:

q_0 : the start configuration, where the tree is rooted

q_g : the goal configuration

n : number of attempts to expand the tree

Result:

A tree $T = (V, E)$ that is rooted at q_0 and has $\leq n$ configurations

$V \leftarrow \{q_0\};$

$E \leftarrow \{ \};$

for $i = 1$ **to** n **do**

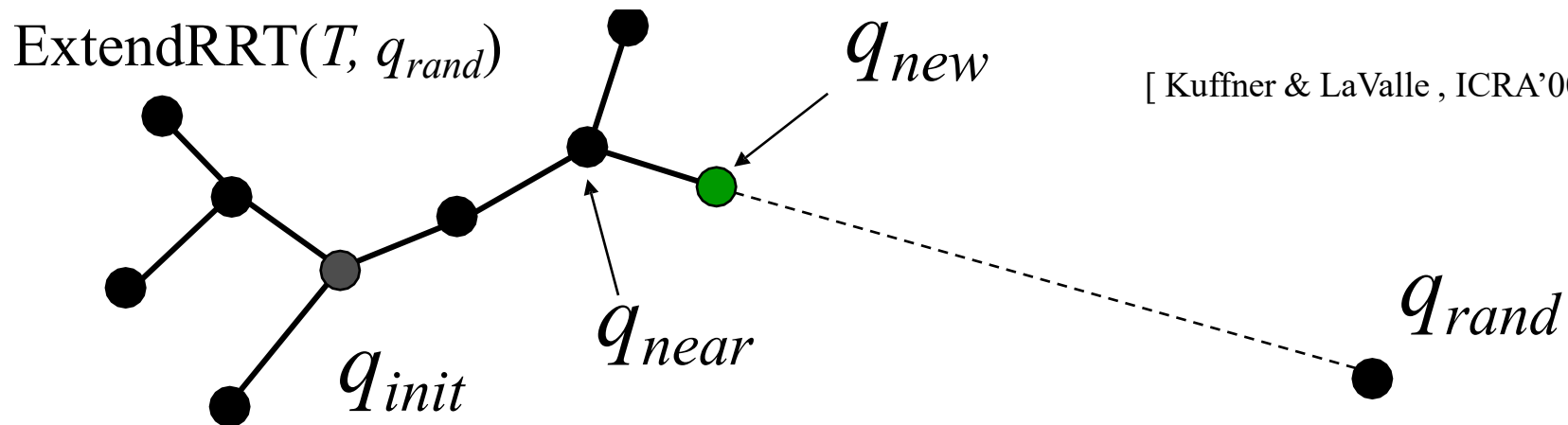
$q_{rand} \leftarrow$ a randomly chosen free configuration;

 ExtendRRT(T, q_{rand});

if $RRTConnect(T, q_g) == true$ **then**

 return T ;

return T ;



[Kuffner & LaValle , ICRA'00]

Motion Planning with RRTs (stepsize)

Algorithm 4: ExtendRRT Algorithm

Data:

$T = (V, E)$: an RRT

q : a configuration toward which the tree T is grown

Result:

A new configuration q_{new} toward q , or NIL in case of failure

$q_{near} \leftarrow$ closest neighbor of q in T ;

$q_{new} \leftarrow$ progress q_{near} by *stepsize* along the straight line in Q between q_{near} and q_{rand} ;

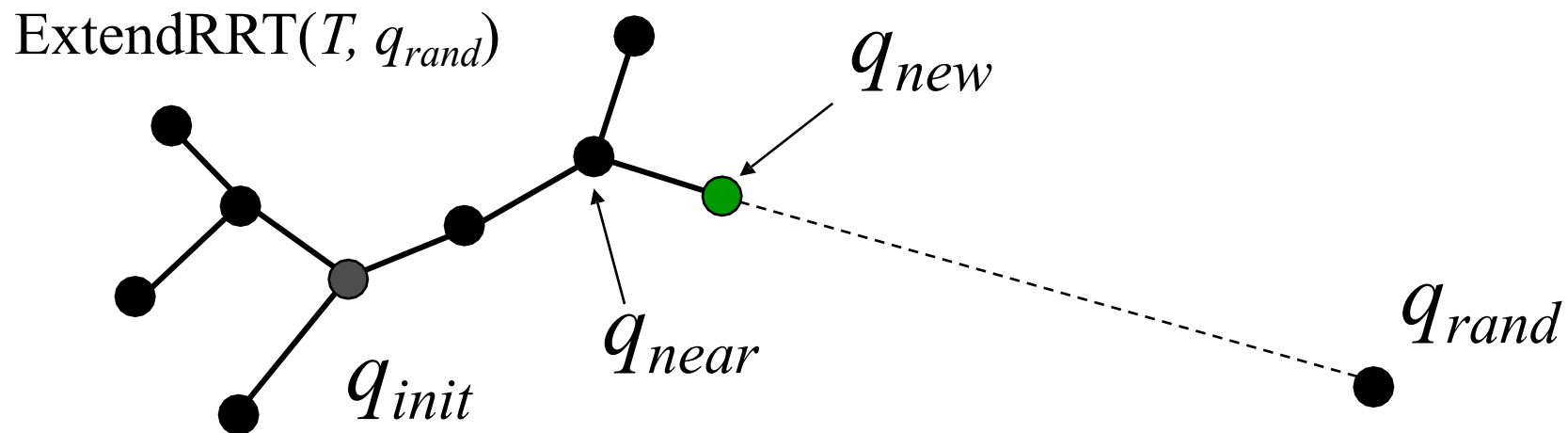
if q_{new} *is collision-free* **then**

$V \leftarrow V \cup \{q_{new}\}$;

$E \leftarrow E \cup \{(q, q_{new})\}$;

return q_{new} ;

return NIL;



Connection to goal

Algorithm 5: Connect RRT Algorithm (ConnectRRT)

Data:

$T = (V, E)$: an RRT

q : a configuration toward which the tree T is grown

Result:

connected if q is connected to T ; *failure* otherwise

repeat

$q_{new} \leftarrow \text{ExtendRRT}(T, q)$;

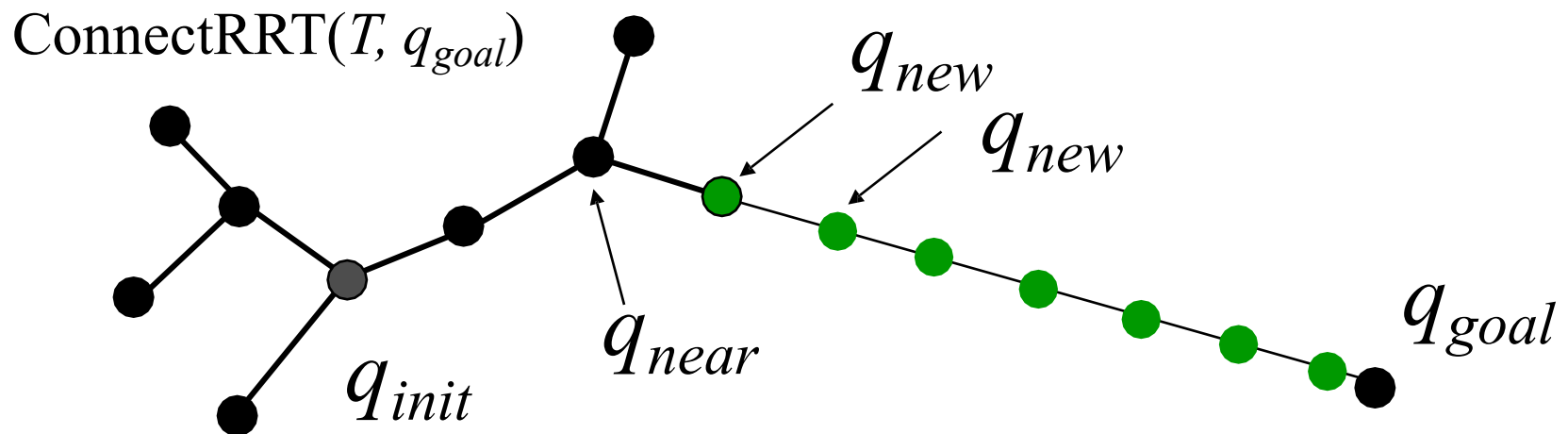
until $q_{new} = q$ or $q_{new} = \text{NIL}$;

if $q_{new} = q$ **then**

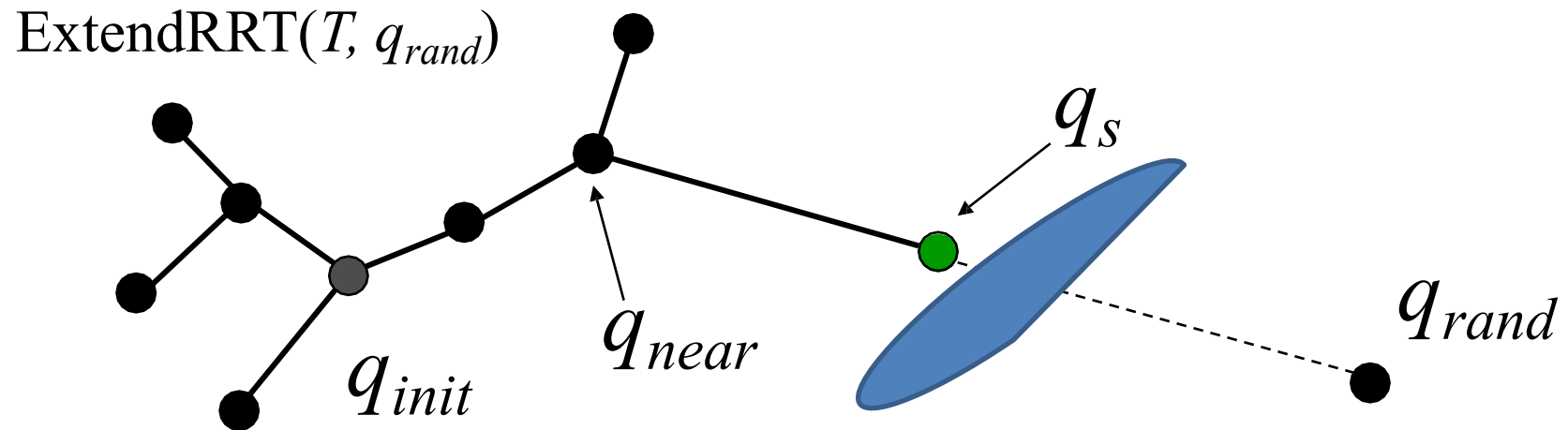
 return true;

else

 return false;

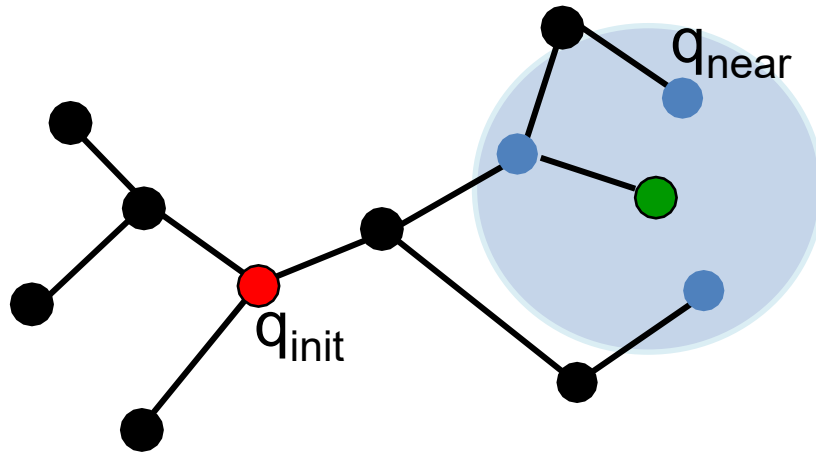
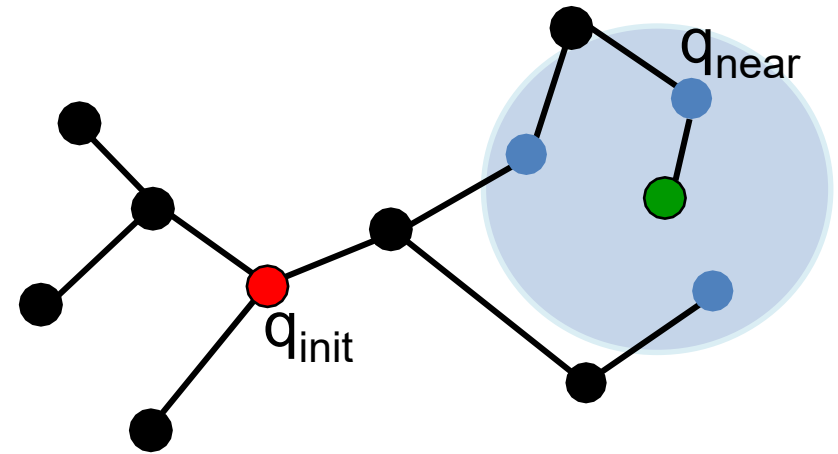
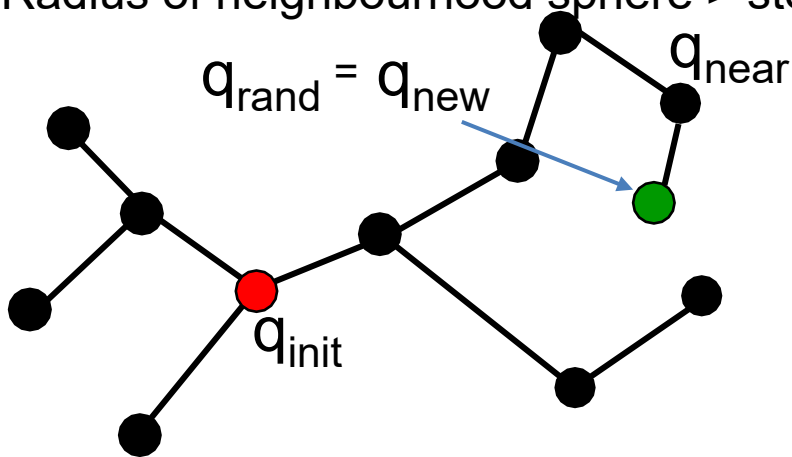


Path Planning with RRTs (stopping configuration)

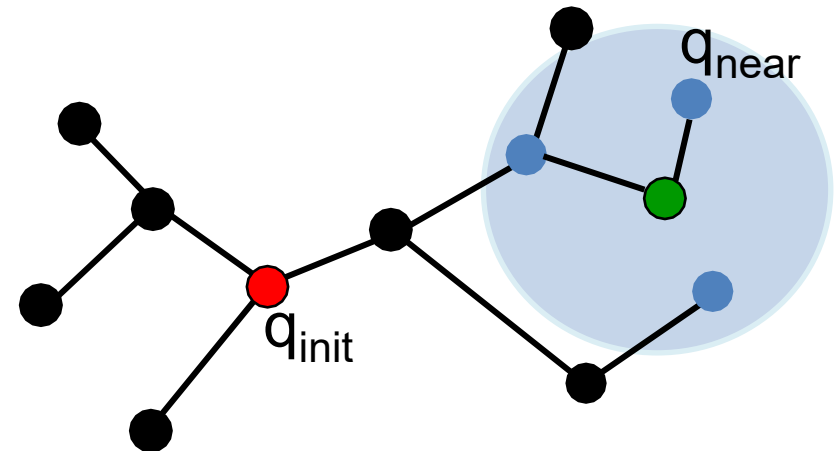


Motion Planning with RRTs (RRT*)

- Stepsize: Small step toward q_{rand}
- Neighbourhood: cost minimization and rearrangement of tree
- Radius of neighbourhood sphere $>$ step size



cost minimization step 1



cost minimization step 2

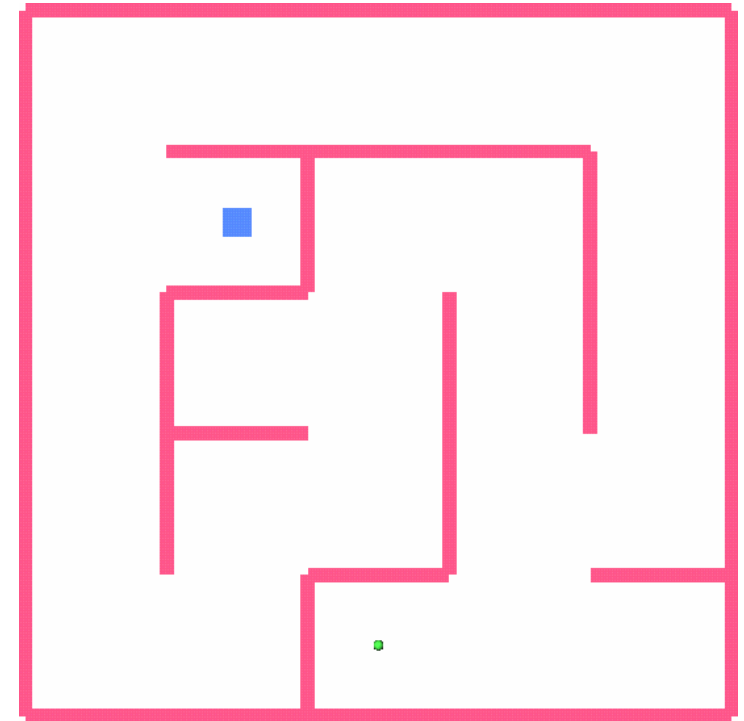
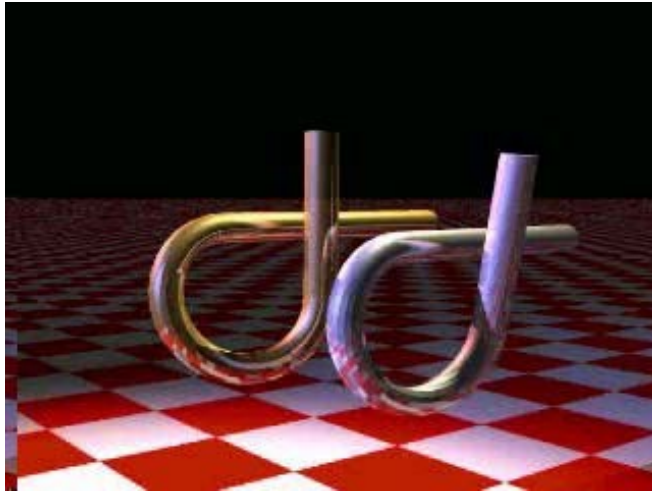
Biases

- Non-biased RRT explores in all directions
 - Not aimed at the goal
- Bias toward goal
 - When generating a random sample, with some probability pick the **goal** instead of a **random** node when expanding
 - Use a small percentage of targets to be the goal or its neighborhood
 - for example. 1% - 5%

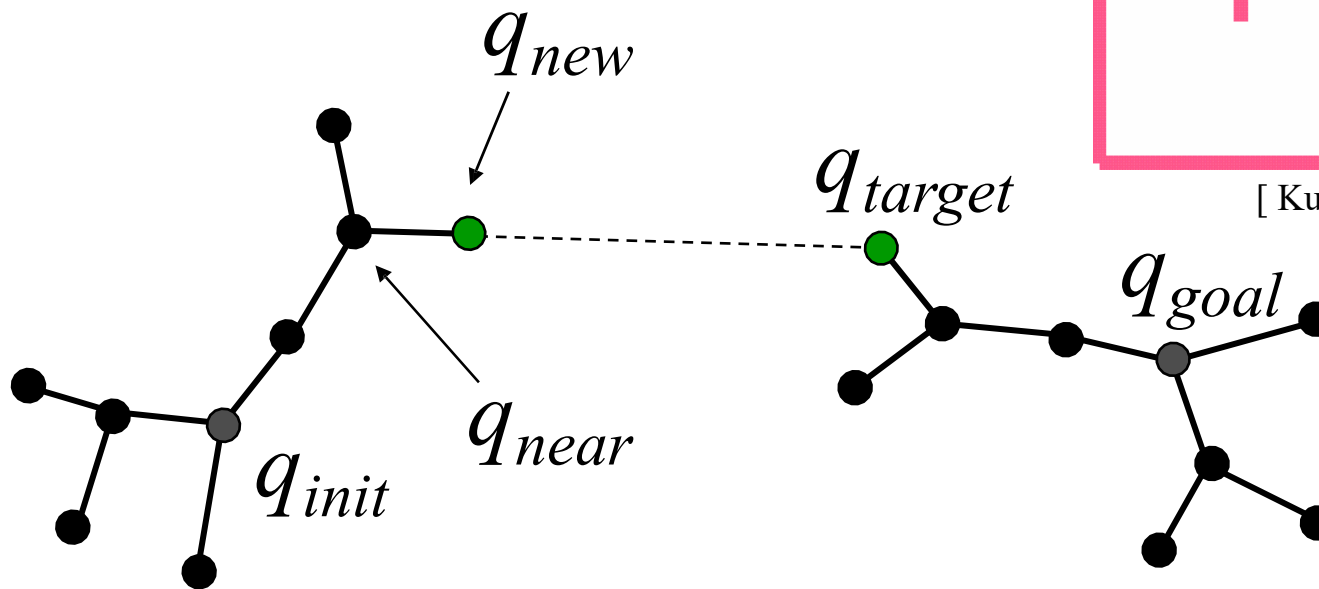
```
RDT( $q_0$ )
1   $\mathcal{G}.\text{init}(q_0);$ 
2  for  $i = 1$  to  $k$  do
3       $q_n \leftarrow \text{NEAREST}(S, \alpha(i));$ 
4       $q_s \leftarrow \text{STOPPING-CONFIGURATION}(q_n, \alpha(i))$ 
5      if  $q_s \neq q_n$  then
6           $\mathcal{G}.\text{add\_vertex}(q_s);$ 
7           $\mathcal{G}.\text{add\_edge}(q_n, q_s);$ 
```



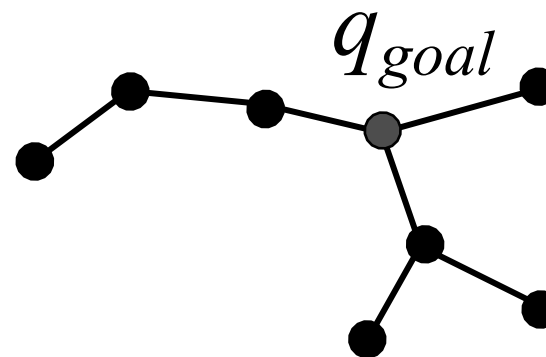
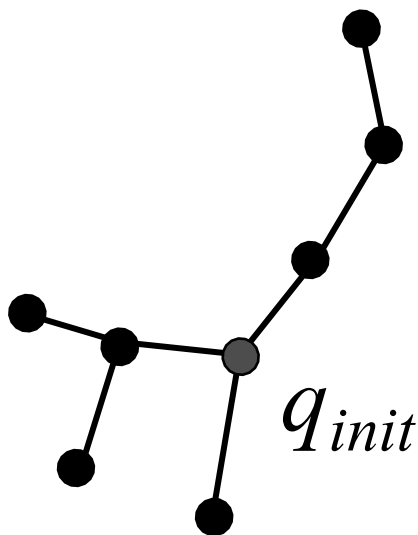
Grow two RRTs towards each other



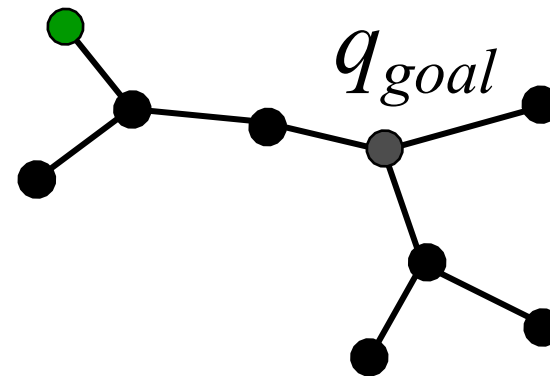
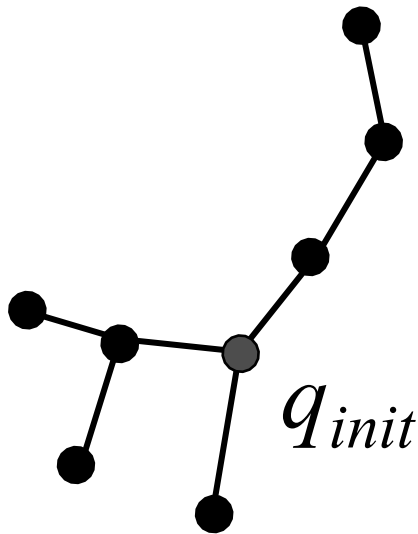
[Kuffner, LaValle ICRA '00]



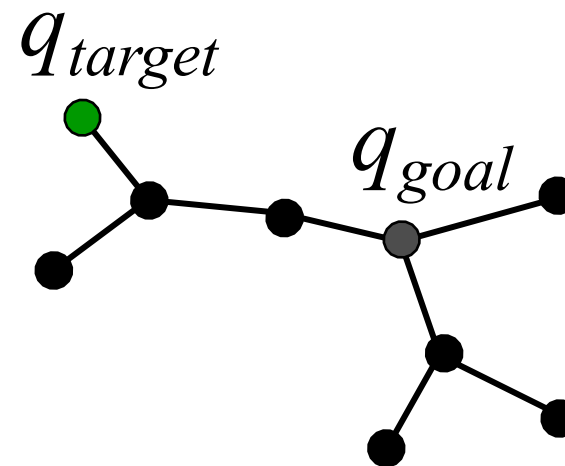
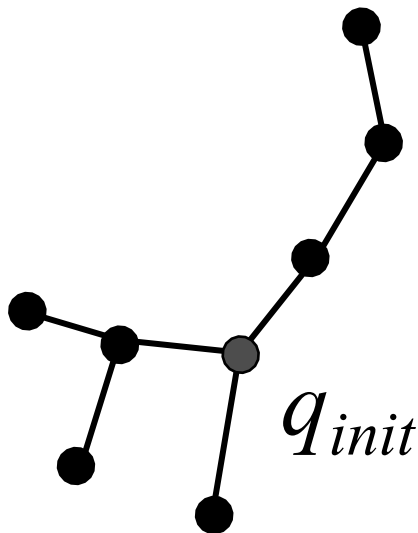
A single RRT-Connect iteration...



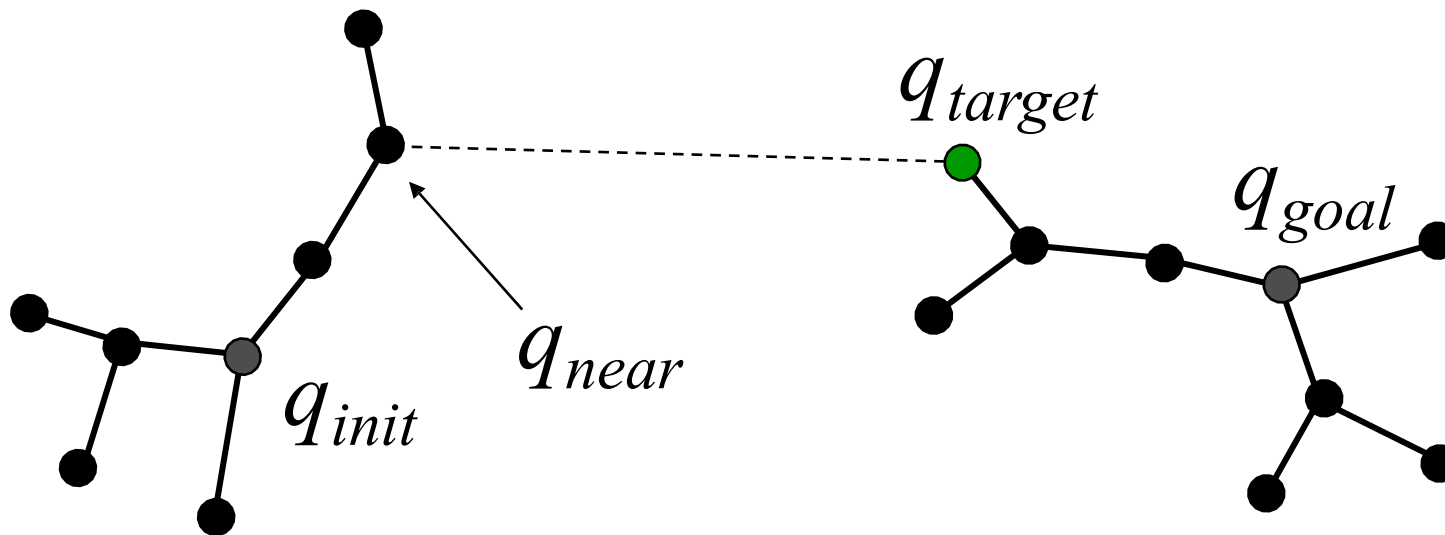
1) One tree grown using random target



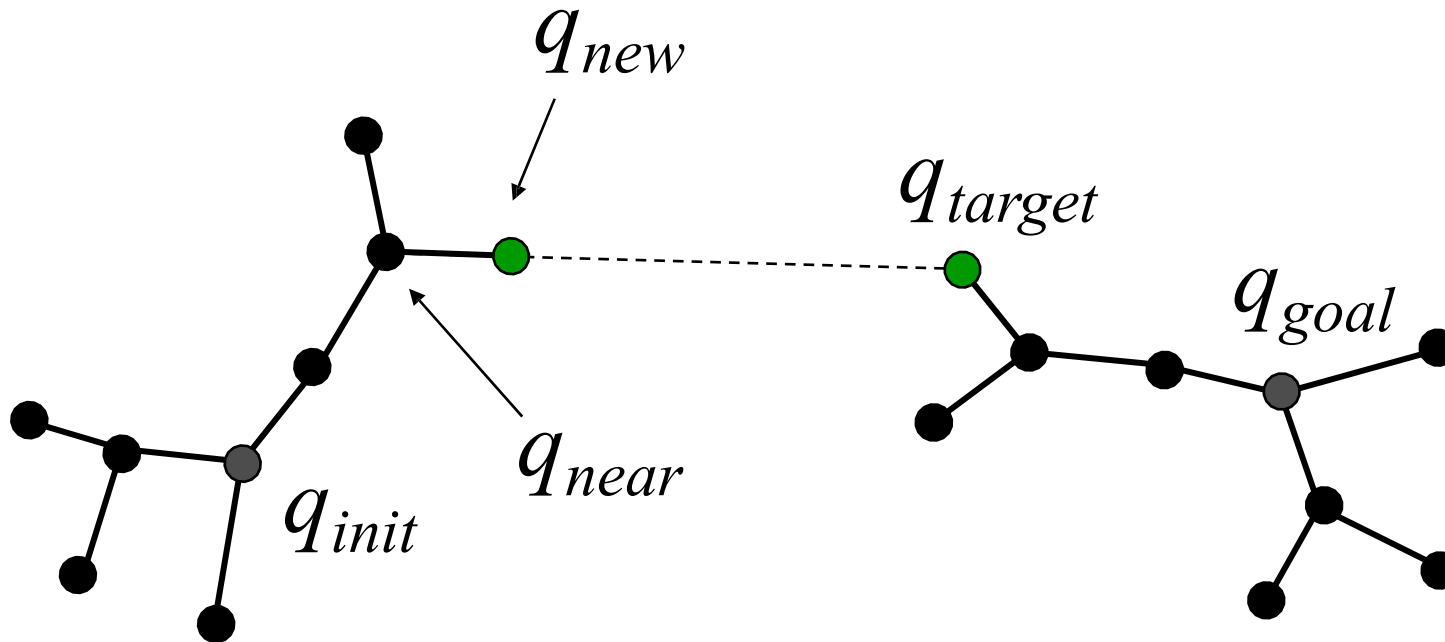
2) New node becomes target for other tree



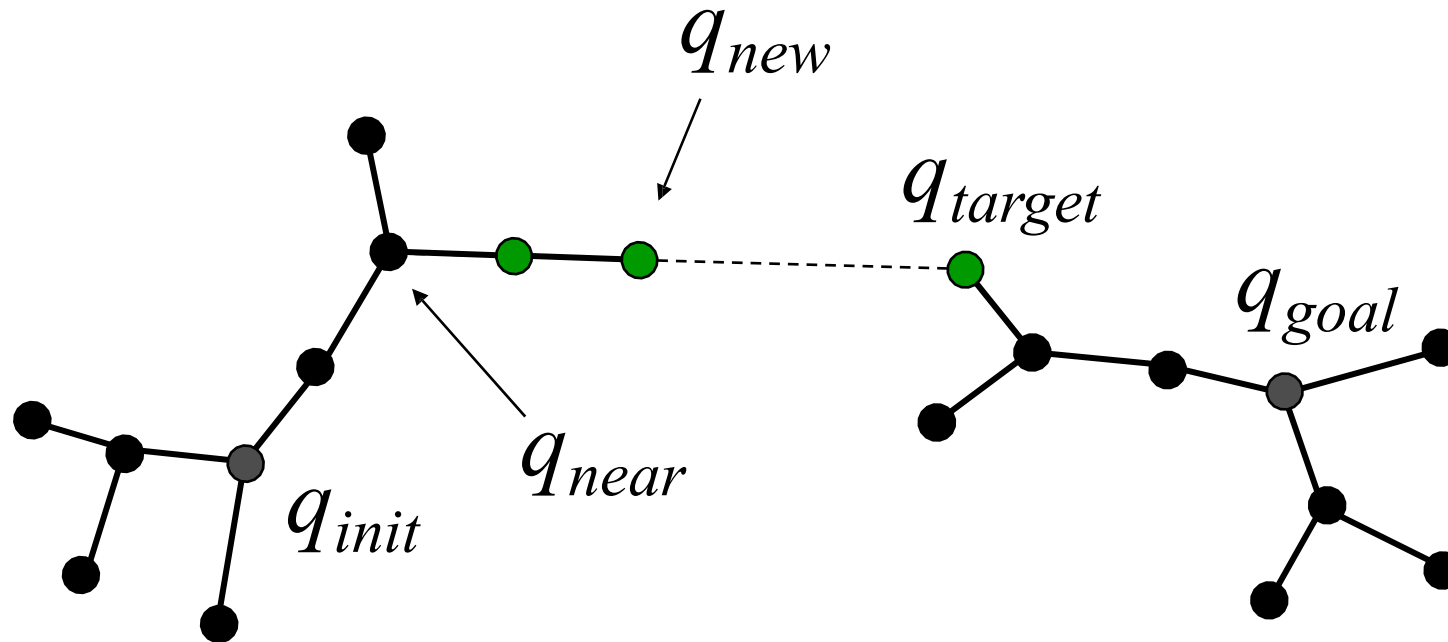
3) Calculate node “nearest” to target



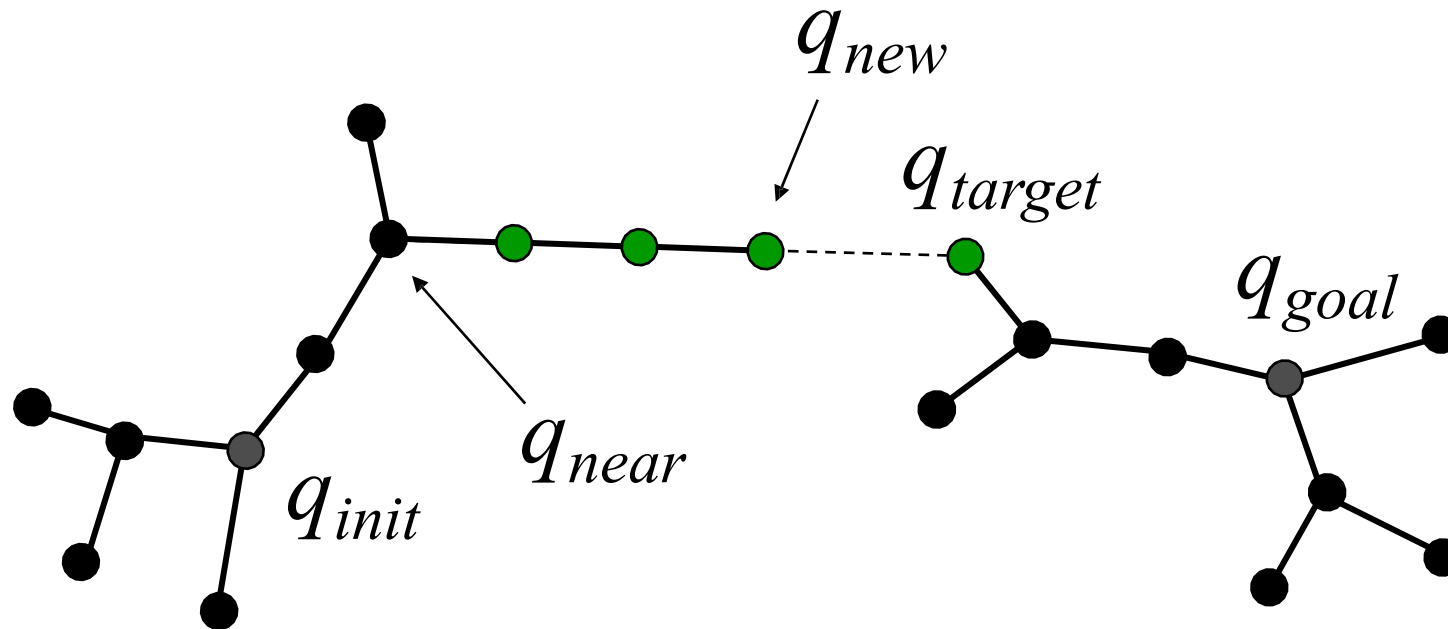
4) Try to add new collision-free branch



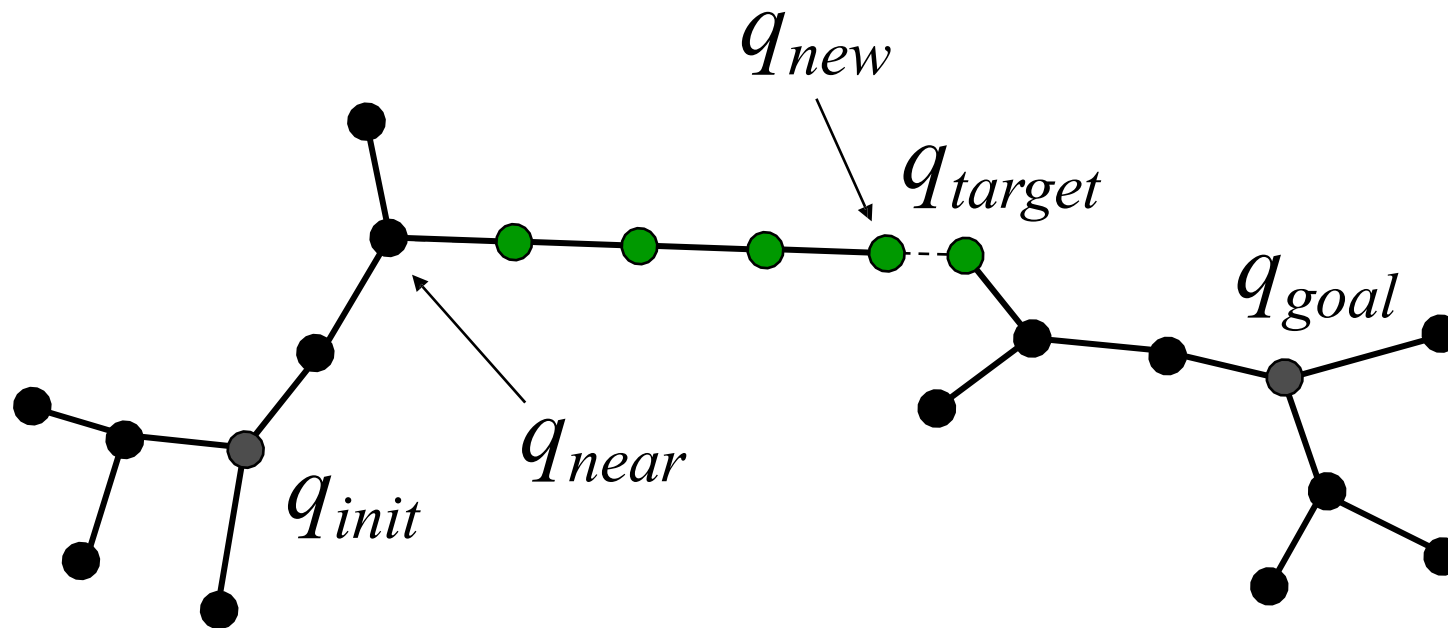
5) If successful, keep extending branch



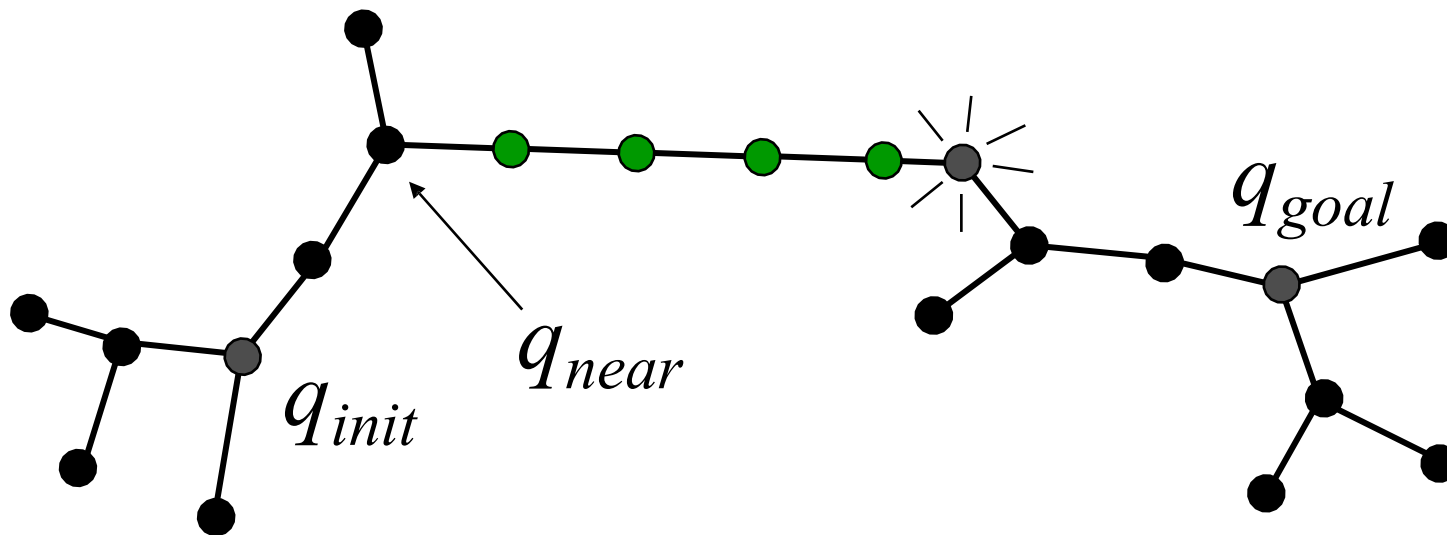
5) If successful, keep extending branch



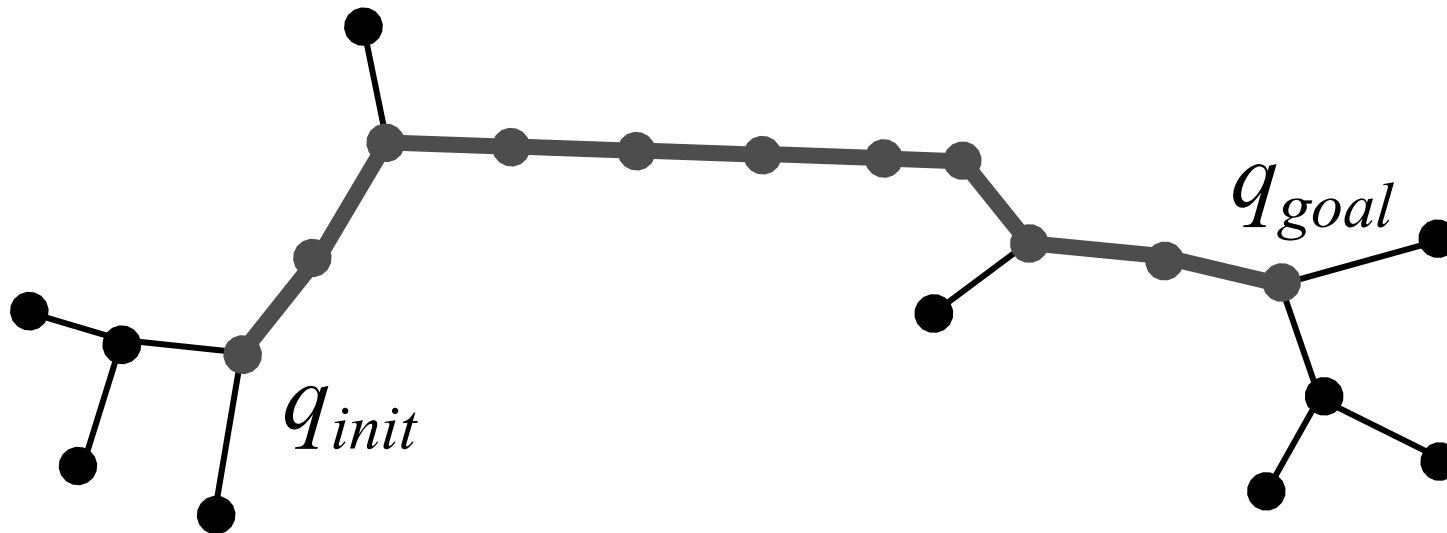
5) If successful, keep extending branch



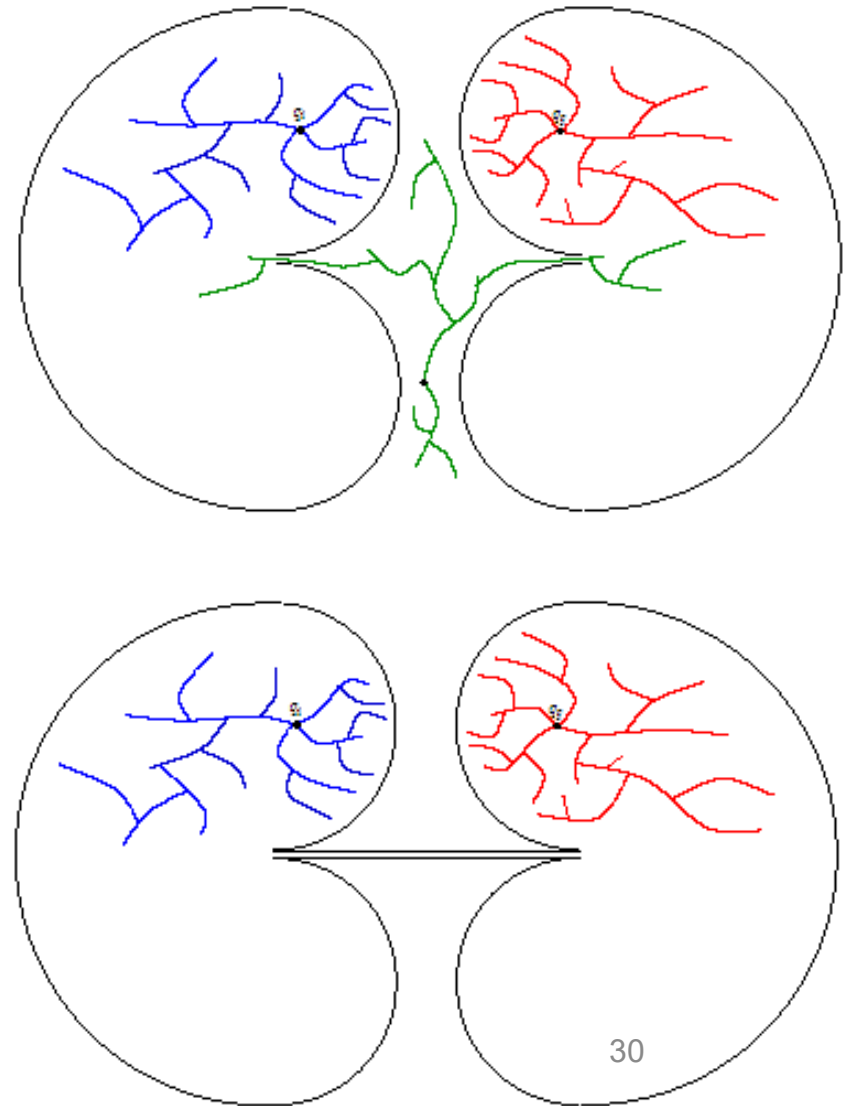
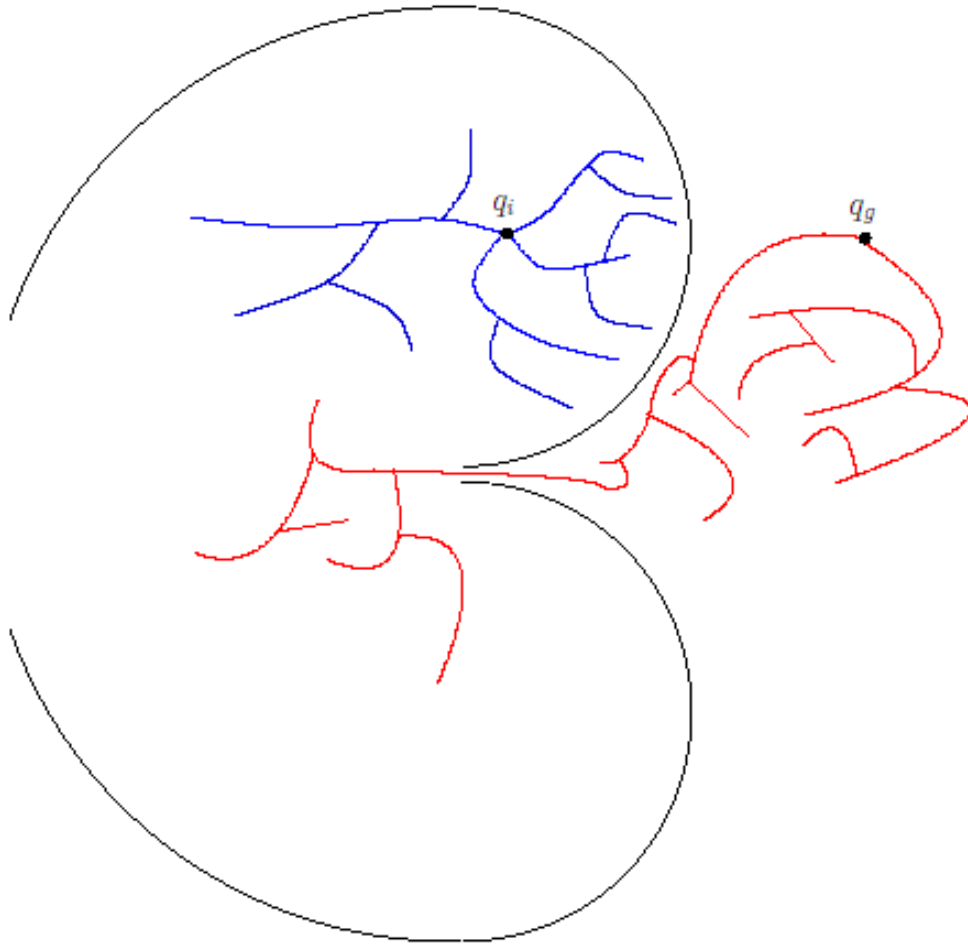
6) Path found if branch reaches target



7) Return path connecting start and goal



Bug traps motivates bidirectional



Basic RRT-Connect

Algorithm 6: Merge RRT Algorithm

Data:

T_1 : first tree

T_2 : second tree

l : a number of attempts allowed to merge T_1 and T_2

Result:

merged if two RRTs are connected to each other; *failure* otherwise

for $i = 1$ **to** l **do**

$q_{rand} \leftarrow$ a randomly chosen free configuration;

$q_{new,1} \leftarrow \text{ExtendRRT}(T_1, q_{rand});$

if $q_{new,1} \neq \text{NIL}$ **then**

$q_{new,2} \leftarrow \text{ExtendRRT}(T_2, q_{new,1});$

if $\text{ConnectRRT}(T_1, q_{new,2}) == \text{true}$ **then**

 return *merged*;

 SWAP(T_1, T_2);

return *failure*;

Applications of RRTs

- Robotics Applications
 - mobile robotics
 - manipulation
 - humanoids
- Other Applications
 - biology (drug design)
 - manufacturing and virtual prototyping (assembly analysis)
verification and validation
 - computer animation and real-time graphics
 - aerospace
- RRT extensions
 - discrete planning (STRIPS and Rubik's cube)
 - real-time RRTs
 - anytime RRTs
 - dynamic domain RRTs
 - deterministic RRTs
 - parallel RRTs
 - hybrid RRTs

RRT Summary

Advantages

- Single parameter
- Balance between greedy search and exploration
- Converges to sampling distribution in the limit
- Simple and easy to implement

Disadvantages

- Metric sensitivity
- Nearest-neighbor efficiency
- Unknown rate of convergence

Links to Further Reading

- Steve LaValle's online book:
"Planning Algorithms" (*chapters 5 & 14*)
<http://lavalle.pl/>
- Youtube Video: RRT, RRT* & Random Trees
<https://www.youtube.com/watch?v=Ob3BIJkQJEw>

Demo: <https://demonstrations.wolfram.com/RapidlyExploringRandomTreeRRTAndRRT/>

- Nice video of Steve M. Lavalle about how he came to idea deveoping RRT's and some entertaining insights about his life at university and industry
<https://www.youtube.com/watch?v=OjNFjruZgaw>